1. PROJE VİZYONU VE AMACI

Bu ödevin temel amacı, Google'ın modern standardı olan **Bildirimsel Arayüz (Declarative UI)** yaklaşımını, **Tek Yönlü Veri Akışı (Unidirectional Data Flow - UDF)** prensiplerini ve **RAM/CPU dostu gelişmiş bellek yönetimi** mekanizmalarını tek bir Kotlin projesinde hayata geçirmektir.

Öğrencilerden, bir altyapı/sistem mühendisi vizyonuyla sunucu kümelerini (Cluster) ve üst düzey yazılım donanımlarını simüle eden, **performans optimizasyonu yapılmış** bir kaynak yönetim paneli (**DevCluster: Donanım ve Sunucu Kaynak Yönetim Simülatörü**) geliştirmeleri beklenmektedir.

2. MİMARİ VE TEKNİK GEREKLİLİKLER

Ödevin geçer not alabilmesi için aşağıda belirtilen dört ana teknik katmanın eksiksiz ve kurallara uygun olarak kodlanması zorunludur:

A. Veri Katmanı (Data Layer) ve Derleyici Sözleşmesi

- **Değişmezlik (Immutability):** Her bir donanım kaynağını temsil eden HardwareItem adında bir data class oluşturulmalıdır. Çok iş parçacıklı (Multithreaded) asenkron süreçlerde *Race Condition* hatalarını engellemek amacıyla sınıfın tüm alanları istisnasız val anahtar kelimesiyle tanımlanmalıdır.
- **Kararlılık Sözleşmesi:** Jetpack Compose derleyicisinin akıllı atlama (*Smart Recomposition Skipping*) motorunu tetikleyebilmesi için veri sınıfının tepesi **@Immutable** anotasyonu ile işaretlenmelidir.

B. Durum Yönetimi ve Donanımsal Koruma (State Management)

- **Çalışma Zamanı Hafıza Kilidi:** Toplam harcanan ödenek miktarı gibi ekranın dinamik durumları, Compose'un doğrusal hafıza yapısı olan *Slot Table* içerisine remember makrosuyla kilitlenmelidir.
- **Konfigürasyon Değişim Koruması:** Kullanıcının arama çubuğuna (TextField) harf yazdığı esnada cihazı yan çevirmesi (Portrait → Landscape) durumunda, Android işletim sisteminin Activity'yi yok edip baştan kurma krizine karşı arama durumu **rememberSaveable** ile RAM dışı geçici *Bundle* alanında muhafaza edilmelidir. Ekran döndüğünde arama satırında veri kaybı yaşanması doğrudan kalma sebebidir.
- **Turnike Mantığı (Memoization):** Arama veya kategori filtreleme algoritmaları, her Recomposition tetiklenmesinde boş yere CPU tüketmemesi için **remember(searchQuery)** önbellek filtresi arkasına alınmalıdır. Algoritma sadece anahtar kelime değiştiğinde çalışmalıdır.

C. Liste Mimarisi ve GPU Hücre Dönüşümü

- **Tembel Yükleme:** Donanımların ekranda akıcı listelenmesi için kesinlikle standart forEach döngüleri kullanılmamalı, **LazyColumn** mimarisi entegre edilmelidir.
- **Nokta Atışı Recomposition:** Listenin ortasından bir eleman güncellendiğinde ya da silindiğinde tüm ekranın baştan çizilmesini önlemek amacıyla items bloğuna benzersiz kimlik yeteneği sunan **key = { it.id }** parametresi zorunlu olarak geçilmelidir.

D. Mimari Ayrışma Disiplini (Stateless UI)

- **Saf Fonksiyon Tasarımı:** Ekranda listelenen donanım kartları (HardwareCard) ve arama çubuğu (HardwareSearchBar) tamamen **Stateless** tasarlanmalıdır. İçlerinde hiçbir yerel remember hücresi barındırmamalıdır.
- **State Hoisting:** Tüm bileşenler durumları yukarıdan parametre olarak almalı (**State Down**), oluşan tıklama veya yazma olaylarını ise Lambda Callbacks (**Events Up**) ile en tepedeki tek bir merkezi Stateful kapsayıcıya fırlatmalıdır.
- **Modifier Sıralaması:** Bileşenlerin Modifier zincirlemelerinde sıralama kuralına dikkat edilmelidir. Örnek olarak, kart bileşenlerinde önce .clickable(), ardından .padding() hiyerarşisi işletilerek görsel margin hatalarının önüne geçilmelidir.

Uygulamanızın test edileceği sabit veri kümesi aşağıdadır. Arama ve filtreleme mekanizmalarınız bu liste üzerinde koşturulacaktır.

listOf(
HardwareItem(id = "101", name = "MacBook Pro M3 Max", category = "Geliştirici Bilgisayarı", cost = 3499.0, stockCount = 5),

HardwareItem(id = "102", name = "NVIDIA RTX 4090 Sunucu", category = "Yapay Zeka / GPU", cost = 1599.0, stockCount = 2),

HardwareItem(id = "103", name = "Google Pixel 8 Pro", category = "Mobil Test Cihazı", cost = 999.0, stockCount = 8),

HardwareItem(id = "104", name = "Dell UltraSharp 49\"", category = "Monitör", cost = 1199.0, stockCount = 4),

HardwareItem(id = "105", name = "Sony WH-1000XM5 Kulaklık", category = "Odaklanma Ekipmanı", cost = 399.0, stockCount = 12),
HardwareItem(id = "106", name = "Raspberry Pi 5 8GB", category = "IoT Geliştirme Kiti", cost = 85.0, stockCount = 25),

HardwareItem(id = "107", name = "HHKB Professional Keyboard", category = "Ergonomik Klavye", cost = 299.0, stockCount = 0), // Stok sınırı test verisi

HardwareItem(id = "108", name = "AMD Ryzen Threadripper", category = "Yapay Zeka / GPU", cost = 4999.0, stockCount = 1))

4. TESLİM VE GİTHUB KONTROL KURALLARI

1. Android Studio üzerinde projenizi oluştururken proje adını tam olarak **week2_assignment** olarak belirleyin.
2. Projenizi kendi kişisel GitHub hesabınızda kamuya açık (**Public**) bir depo (repository) olarak yayımlayın.
3. Projenizin app/src/main/java/com/example/week2/MainActivity.kt dosyasının eksiksiz ve erişilebilir olduğundan emin olun.
4. Hazırladığınız deponun (repository) web linkini, ödev yükleme alanına son teslim tarihine kadar metin olarak yapıştırıp gönderin.